Module 1: Formal Syntax and Semantics

Topics

1. Introduction
2. Formal Syntax
3. Static Semantics
4. Dynamic Semantics
5. Syntax Translation

I. Introduction

Although our primary focus in this module is formal syntax and semantics, we begin with several introductory topics, which include the criteria by which we plan to evaluate the features of programming languages, an introduction to language translators, and a brief history of programming languages.

A. Evaluating Language Design

Throughout these modules, we will compare the ways in which different programming languages implement various language features. As a part of the comparison, we will often make judgments regarding which implementation is more desirable. To be able to make such judgments, we must establish a set of criteria to use.

Most language critics would agree that programming languages should be easy to read, easy to write, and capable of producing error-free programs. There is less agreement, however, on the way in which specific language features promote these three goals. What complicates language evaluation is that when we look at more specific language attributes such as simplicity or expressivity—both desirable attributes—what enhances one may lessen the other. Specifically, a language that is too simple may be less expressive.

As we examine language features, we will pay close attention to relationships between them. One of the most undesirable relationships is feature interaction that creates ambiguity, or worse, the possibility of incorrect programs. The latter occurs when two or more features result in programs that would reasonably be expected to do one thing but instead do something else, possibly something that is not at all meaningful. The following statement in either C or C++ illustrates an example of this kind of feature interaction:

```
if (x = 0)
  ...
```

Three features contribute to the misinterpretation of this statement. The first is the similarity of the assignment operator and the relational operator for equality and the fact that in other languages this operator is a relational operator. The second is the fact that assignments are expressions, not statements, as they are in some languages. The third feature is the fact that C and C++ have no true Boolean data type. In module 2, when we discuss expressions, we will see how all three features must be present to create this misinterpretation.

Another important relationship between language features is whether or not they are *orthogonal*. Features are orthogonal when all possible combinations of the two features are allowed. An example of two orthogonal features in C or C++ is whether integral data types are signed or unsigned. Table 1.1 illustrates the orthogonality of these two features:

Table 1.1
Orthogonality of Data-Type Size and Signedness

	char	short	int	long
unsigned	√	√	√	√
signed	√	√	√	√

If, for some reason, some combinations of these two features were not permitted, we would regard them as not orthogonal.

Another important relationship between features is whether two features are redundant, sometimes referred to as *feature multiplicity*. The C or C++ while and for statements are redundant features. These statements are completely interchangeable, meaning that wherever one could be used the other could be used instead. In other cases of feature multiplicity, one feature may be more general than the other. The latter situation is usually more desirable. We will explore why that is so when we encounter our next instance of redundant features.

As we explore various language features, we may find it necessary to use other criteria for comparison. When making judgments about the quality of language design, there is considerable room for opinion, no matter how specific we try to be with our evaluation criteria. Bear this in mind because it is not the case with many aspects of computer science.

B. Language Translators

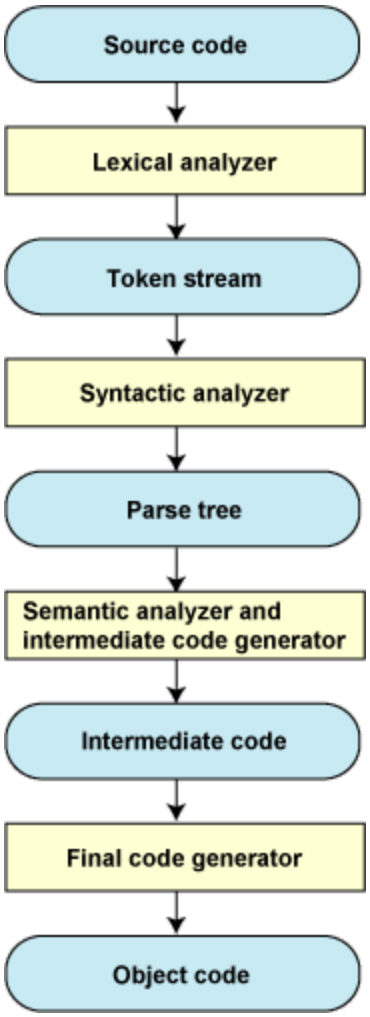
The design of programming languages is intimately connected with the design of the translators that translate programs written in a particular language into a lower-level language. Consequently, understanding the way in which language-design decisions influence the complexity of language translators requires some understanding of how translators are built. The decision of whether a language will support nested subprograms greatly influences the complexity of its compiler, which is one kind of translator. In module 3, we will see how much more complex a compiler must be when the language includes such subprograms.

We will discuss three types of translation mechanics: full compilation, full interpretation, and hybrids of the two.

Compiled Languages

C, C++, Pascal, Ada, and many other languages are fully compiled. Compilers for such languages accept the source code and produce object code. Figure 1.1 illustrates the phases of such a compiler.

Figure 1.1
Phases of a Compiler



Most compilers consist of two passes. The first pass, which transforms the source code to intermediate code, is machine independent. The second pass, which produces the object code, is not. Most compiled languages support separate compilation. For programs that consist of multiple source files, the object files corresponding to those source files must be linked together with a program called a *linker*. Regardless of whether there are multiple source files, the object code must be linked with the object library, which contains common code used by all programs, such as code to perform input and output.

Interpreted Languages

Languages that have a relatively simple syntax and little or no data typing tend to be fully interpreted. LISP, a language we will study in module 5, is one language that is generally interpreted. Early versions of BASIC are another example. The difference between an interpreter and a compiler is that an interpreter translates and executes the program, whereas a compiler only performs the translation.

There are tradeoffs between these two alternatives. The advantage of full compilation is that the programs will run faster because the translation is only done once, not each time the program is run. Debugging programs written in fully interpreted languages was advantageous—especially when compilers were slow. Because most compilers today run so fast, the benefit of full interpretation when debugging has diminished.

Hybrid Languages

Finally, some languages are partly compiled and partly interpreted. This technique was used with early versions of Java. The Java compiler translates the source into intermediate code, which for Java is called *bytecode*. The interpreter, which is referred to as the *Java Virtual Machine* (JVM), translates and executes the bytecode. More recently, just-in-time (JIT) compilers have replaced the interpreter. A JIT compiler, itself, is a kind of hybrid. It translates and executes like an interpreter, but like a compiler, it ensures that the translation is done only once.

C. History of Programming Languages

Although the history of programming languages is relatively short, given the number of programming languages that have been developed during the past half-century, it is a rich history. Rather than attempt to describe all or even a significant number of the languages that have been developed, we instead elect to describe several significant steps in the evolution of languages. This characterization will be somewhat chronological, but not entirely so. Our focus instead will be on the level of abstraction of languages.

Machine-Level Languages

Historically, unstructured imperative languages were the languages used first. *Machine language*, which consists of pure binary numeric code, and *assembly language*, a symbolic equivalent of machine language, are examples. Such languages are the least abstract, which means they are closest to the hardware. They are the languages into which all higher-level languages must eventually be translated. What is important to understand about such languages is that they have no nesting—either at the level of expressions or the level of statements. Each line of code in such languages has at most one operator, and control flow is achieved with unconditional or conditional *goto*-like instructions.

High-Level Imperative Languages

What separates high-level languages from machine language is the presence of nesting, which introduces a greater degree of abstraction to programs written in such languages. Nesting really entered programming languages in two steps, although historically these events happened quite close together in time. Fortran introduced nested expressions, which represent mathematical formulas, but the earliest versions of Fortran permitted almost no nesting of statements.

Structured Imperative Languages

ALGOL, which was developed shortly after Fortran, allowed fully nested statements. This ability makes it possible to program without *goto* statements, although it took almost 30 years before popular languages actually removed such statements.

Declarative Languages

Declarative languages, which include both functional and logic languages, are the most abstract. They are closest to mathematics and furthest from the hardware. Functional languages are based on a branch of mathematical logic called *lambda calculus.* Logic languages are based on a branch called *predicate calculus*.

Although most imperative languages to some degree support recursion and function parameters, declarative languages rely on recursion and higher-order functions in place of iteration, variables, and assignments. Programs written in declarative languages specify definitions of the program more than algorithms for their solution.

Because declarative languages differ fundamentally from imperative ones, we will wait until module 5, when we explore them more fully, to try to characterize all the ways in which they differ from imperative languages.

Object-Oriented Languages

Object-orientation incorporated abstraction into programming languages, but a kind that differs from what we have thus far discussed. The movement toward object-orientation early on was often referred to as *data abstraction*.

Object-orientation is orthogonal to whether a language is imperative or declarative. Table 1.2 illustrates this fact by showing one language in each of the four categories.

Table 1.2
Orthogonality of Object-Orientation and Imperative vs. Declarative

	Non-Object-Oriented	**Object-Oriented**
Imperative	C	C++
Declarative	LISP	CLOS

Object-orientation shifts the focus of design from control structure to data structure, although the data units are objects, which are compound data structures, not primitive variables.

II. Formal Syntax

We now turn to the primary topic for this module—formal syntax and semantics. Like natural languages, programming languages have syntax, which is the grammar, and semantics, which is the meaning. The first two phases of the compiler determine whether the program supplied to the compiler is syntactically correct. Syntax is sometimes divided into two levels, the *lexical level*, which is checked by the lexical analyzer, and the *textual level*, which is checked by the syntactical analyzer.

A. Regular Expressions

Specifying the syntax at the lexical level is synonymous with defining the syntax of the tokens of the language. Regular expressions are generally used to specify syntax at this level. It is easiest to explain how regular expressions are used with an example. Shown below is the regular expression that defines the identifier token in Ada.

```
[a-zA-Z](_?[a-zA-Z0-9])*
```

In Ada, identifiers must begin with a letter followed by zero or more letters or digits. Underscores are permitted, but leading, trailing, and consecutive underscores are prohibited. Clearly, the definition that uses a regular expression is more concise than the equivalent English definition and usually more precise because English can often be unclear or ambiguous. Of course, to understand the regular expression, it is important to understand the following metasymbols

- The ? indicates optional.
- The * means zero or more.
- The - specifies a range of allowable characters.
- The square brackets define a collection of characters from which one can be chosen.
- The parentheses act as a grouping mechanism.

Although the language of regular expressions is not a programming language, it is a declarative language, in that it is used to define language syntax, and it is a metalanguage

Regular expressions are a less powerful language-defining mechanism than the context-free grammars that we will encounter shortly. In particular, regular expressions cannot be used to describe syntax that involves nesting. Consequently, most programming-language token definitions do not permit nesting. In particular, comments cannot be nested in C, as the unsuccessful attempt below illustrates:

```
/* /* An attempt to nest comments */ */
```

In the above example, the comment ends at the first */.

B. Backus-Naur Form

To describe the syntax of a programming language at the textual level, a more powerful metalanguage called *Backus-Naur Form* (BNF) is used. Using BNF, we write a grammar that consists of a collection of productions. The productions consist of terminal symbols, nonterminal symbols, and punctuation metasymbols.

Every production contains the metasymbol → that separates the left-hand side of the production from the right. That symbol is read as "can be replaced by" or "is defined as." The left-hand side of every production consists of a single nonterminal, which represents the nonterminal being defined, and the right-hand side consists of a collection of nonterminals and terminals. As an example of a production, consider the production that defines the syntax of a while statement in C, C++, or Java:

```
<while_stmt> → while (<exp>) <stmt>
```

The reserved word while and the parentheses are terminals, whereas <exp> and <stmt> are nonterminals.

The metasymbol | can be used to condense multiple productions with a common left-hand side into a single production. The metasymbol | separates various choices for the right-hand side. Extended BNF allows additional metasymbols.

Although extended BNF allows the metasymbols {} to be used for zero or more occurrences, in standard BNF, recursive productions are needed to define components that occur multiple times.

Shown below is an example of a recursive production that is used to define statement lists in Pascal. Pascal is unusual among languages in that the semicolon is a statement separator rather than a statement terminal, which the following production illustrates:

```
<stmt_list> → <stmt_list> ; <stmt> | <stmt>
```

The above production is left recursive or head recursive. In this case, a right-recursive or tail-recursive production could also have been used. Notice that, like recursive methods, recursive productions must have a base case—a nonrecursive choice.

To determine whether a program is syntactically correct, the parser phase of a compiler attempts to construct a parse tree for the program. The ability to construct a parse tree verifies syntactic correctness. Because the parser phase of

most compilers is not a separate pass, only a small portion of the parse tree is actually maintained at any point in time. Parse trees serve a second purpose, which is that they are used to generate intermediate code.

It is possible to write more than one grammar to define any language. As an example, we could define the Pascal statement lists that we discussed earlier with the following right-recursive production:

```
<stmt_list> → <stmt> ; <stmt_list> | <stmt>
```

Although these two productions define the same syntax for statement lists, the parse trees that would be constructed in the two cases would be different. If all we cared about was using parse trees to verify syntactic correctness, either grammar could be used, but recall we stated earlier that parse trees are also used to generate intermediate code. To generate proper intermediate code, it is important to define a grammar that produces the correct parse tree.

Expression Grammars

Ensuring that our grammar produces the correct parse tree is especially important for expressions. Therefore, next we will consider a variety of different grammars that we might use to define expressions and consider how that choice affects the parse trees that are produced. Let's begin with a relatively simple grammar for expressions in a typical programming language—one that includes the exponential operator.

```
<exp> → <exp> <op> <fact> | <fact>
<fact> → <id> | <cons>
<id> → x | y
<cons> → 1 | 2
<op> → + | - | * | / | **
```

The last three productions of our grammar define the syntax of the identifier, integer literal constant, and operator tokens. Earlier, we mentioned that it is common to use regular expressions for this purpose. It is worth noting, however, that BNF can define any language defined by regular expressions, although the reverse is not true. Notice, too, that our definition for the identifier token is much

more limited than our previous definition. We have elected to use a grammar for both the lexical and textual levels of syntax in this case so that the leaves of our parse tree will be lexemes rather than just tokens.
Figure 1.2 illustrates the construction of the parse tree for one expression. The parse tree is constructed from the top down using a left-most derivation.

Figure 1.2
Construction of a Left-to-Right Parse Tree

0:00 / 2:41

Viewing the completed tree from the bottom up, it grows from left to right. It must grow in this fashion because the only recursive production in this grammar is left-recursive. We could change that production to be right-recursive without changing the language, but if we made that change, the parse tree for this expression would grow from right to left.

As we have been emphasizing, it is necessary to have a grammar that will produce parse trees that generate the proper code. If we use the above parse tree to generate intermediate code, we would emit instructions in postorder, giving the following intermediate code:

```
t1 = x + 1
t2 = t1 * y
t3 = t2 ** 2
```

The variables t1, t2,and t3 are temporary variables. We will see more examples using temporary variables later in this module when we discuss operational semantics.

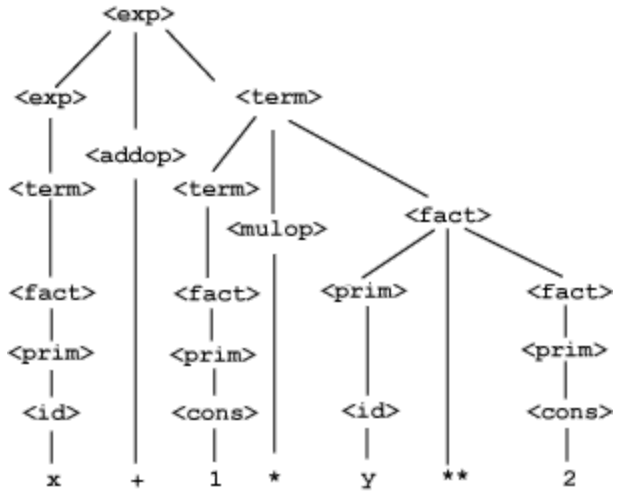
In most programming languages, not all operators have the same precedence—a characteristic that expressions borrow from mathematical formulas. Clearly, the code generated from that parse tree does not follow the usual precedence rules.

By modifying the grammar, we can ensure that a parse tree will be produced that will generate code consistent with the precedence rules. Consider the following expression grammar:

```
<exp> → <exp> <addop> <term> | <term>
<term> → <term> <mulop> <fact> | <fact>
<fact> → <prim> ** <fact> | <prim>
<prim> → <id> | <cons>
<id> → x | y
<cons> → 1 | 2
<addop> → + |-
<mulop> → * | /
```

Figure 1.3 illustrates the parse tree for the same expression used previously. Notice, however, that the parse tree that is constructed will now facilitate generating the proper code.

Figure 1.3
Construction of a Parse Tree According to Operator Precedence



One other thing to notice is that the parse trees generated by this grammar will also properly reflect the associativity rules—the adding and multiplying operators being left associative and exponentiation being right associative—because the recursive rules for the expression and term are left recursive and the recursive rule for a factor is right recursive.

One aspect of typical expression syntax we have thus far omitted in our grammars is the possibility of parentheses. Most languages allow, but do not require, parenthesizing subexpressions, in which case the parentheses override the rules of precedence and associativity.

Adding the following production to our previous grammar will accomplish this goal:

```
<prim> → (<exp>)
```

With the addition of this production, our grammar now contains indirectly recursive productions: an expression can be a term, a term can be a factor, a factor can be a primary, and, finally, a primary can contain an expression.

Ambiguous Grammars

There is one other characteristic of certain grammars that you should understand, which is whether a grammar is ambiguous. None of the grammars we have examined thus far have been ambiguous. Although it is easy to modify them to make them ambiguous, we should caution you that we are doing so only to illustrate ambiguous grammars, not because such grammars are desirable. In fact, we prefer to avoid them.

Consider the following modified expression production of our first expression grammar:

```
<exp> → <exp> <op> <exp> | <fact>
```

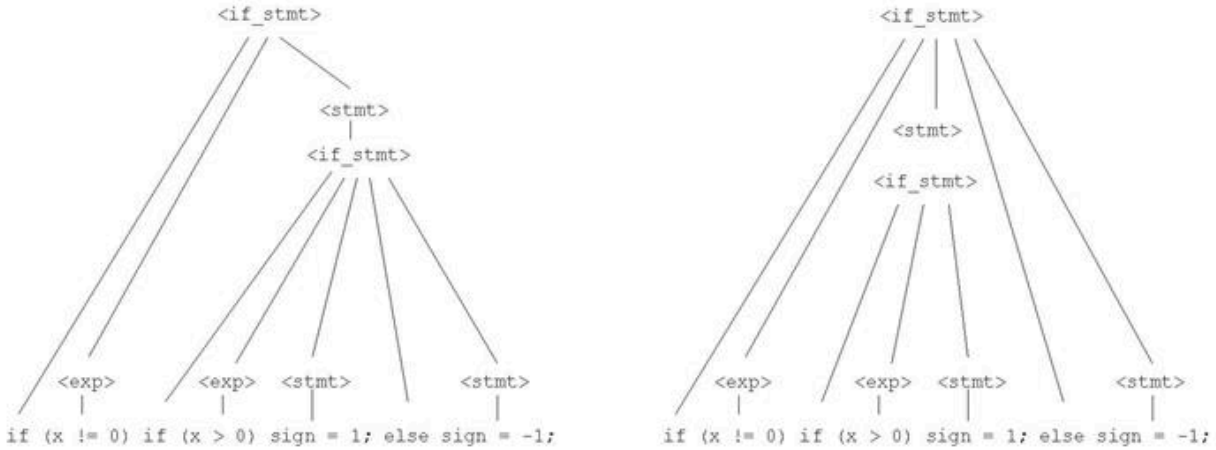
Notice that this production is both left- and right-recursive, which is the cause of the ambiguity. Consequently, with this grammar, parse trees that grow from left to right or right to left, as well as any combination of the two, can be constructed. There are other ambiguous grammars, in which the cause of the ambiguity is more subtle. One of the most important such productions that occurs in grammars of many programming languages is the one that defines the syntax of the if statement, as shown below.

```
<if_stmt> → if (<exp>) <stmt> [else <stmt>]
```

Note also that the above production uses the square bracket metasymbols of extended BNF that denote *optional*. They surround the else clause in the above production.

Figure 1.4 illustrates the ambiguity caused by this production, showing two parse trees for the same statement. Note that for simplicity, the parse trees in this diagram are partial. We have omitted the intermediate details for assignments and expressions to avoid overcomplicating what we are trying to illustrate.

Figure 1.4
Ambiguity of if Statement Grammar



Although it is possible to write a grammar that is not ambiguous, altering the language syntax is the better solution as we will see in module 2 when we discuss the dangling else problem.

III. Static Semantics

For a program to compile, it must satisfy both the syntax rules of the language and the static semantic rules. Whereas the syntax rules verify that a program has correct grammar, the static semantic rules ensure that the program is meaningful.

A. Static Semantic Rules in English

To help you understand this distinction, let's consider the analogous requirements in English. Although you may never have thought about it, English sentences must satisfy both syntax and static semantic rules to be correct sentences. To illustrate this point, consider the following sentence:

The idea ate the house.

Syntactically, this sentence is fine. It is a declarative sentence containing a transitive verb and a direct object, but the sentence is not meaningful. The reason is that although *idea* is a noun, it is not a physical object and, moreover, not an animate one. Furthermore, *house* is also a noun, but not an edible object.

These same two requirements apply at the lexical level—both in English and in programming languages. Let's consider how they apply in English. At the lexical level, there are syntax rules governing how letters form words that are very similar to the regular expression rule that we encountered earlier, which defines the rules for defining identifiers. In English, the rules involve the alternation of consonants and vowels, although certain consonant strings such as *str* are permitted, whereas many others such as *tzl* are not. Other more arcane rules also exist, such as the requirement that the consonant *q* be followed exclusively by the vowel *u*.

Just because a string of English characters follows the syntax rules for English words does not make the string a word. The string *fap* fulfills the syntax rules but not the static semantic ones because that character string has no meaning. To have

meaning, it must be in the dictionary, which is the primary static semantic requirement at the lexical level in English.

B. Static Semantic Rules in Programming Languages

Having considered the significance of static semantics in English, let's now consider how these rules apply to programming languages beginning at the lexical level. In programming languages, at the lexical level, the only static semantic rule applies to identifiers. For an identifier to be meaningful, it must be in the compiler's symbol table, which means that in most languages it either must be predefined or have been defined within the program being compiled.

Next, we consider such rules at the textual level. Let's return to an example we presented earlier when we were discussing feature interaction. The example we used was:

```
if (x = 0)
...
```

We noted that in C and C++, such a statement will compile but will be misinterpreted. The same statement in Java will produce a compilation error, not because it violates any syntax rule, but because it violates a rule that pertains to static semantics. The rule in Java is that the expression in an if condition must have type boolean. Consequently, there are statements with the same syntax that will compile, for example:

```
boolean b;
if (b = true)
...
```

Many rules involving static semantics are related to type checking. Others pertain not to the type of identifiers but to their categories. As an example of the latter, consider the following assignment in C, C++, or Java:

```
PI = 3.14159;
```

Based on the commonly used lexical convention, we infer that PI is a declared constant, so this assignment will not compile. It is because of the rule that the identifier on the left-hand side of the assignment must be the name of a variable and cannot be the name of a constant.

It is the semantic-analyzer phase of the compiler that checks these rules and generates error messages when they are violated. The semantic analyzer also frequently checks rules, sometimes referred to as *context rules*, that are really rules of syntax but awkward to define without excessively complicating the BNF. An example of such a rule is that an EXIT statement in Ada can only appear inside a loop and in no other context.

C. Expressing Static Semantic Rules with Attribute Grammars

BNF grammars can be extended by adding additional rules that involve annotations with attribute information that incorporate static semantics. These extended grammars are known as *attribute grammars*. They contain two additional kinds of rules—one kind to define how attribute information is carried up the parse tree and another kind that specifies under what conditions compilation errors are to be generated.

Because the former kind of rule is mainly of interest to those building compilers, we address only the latter kind of rule, called *predicates*, which are of interest to both compiler writers and compiler users. As our example of a predicate, we provide the one that would be needed to express the rule associated with the required type of the expression inside a Java if statement:

```
expression.type == boolean
```

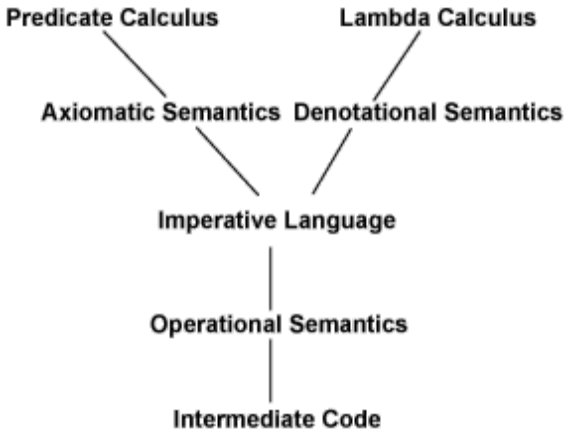
Clearly, the attribute .type refers to the type of the expression. Unfortunately, not all error conditions that are typically generated by the semantic analyzer can be easily expressed with attribute grammars.

IV. Dynamic Semantics

Now that we have discussed the rules that define whether or not a program is meaningful, we consider meaning from another perspective—the way in which the meaning of a program can be described, which is referred to as *dynamic semantics*.

The only way that the meaning of a language can be described is with another language. Each of the various models for dynamic semantics essentially uses some other language. Figure 1.5 illustrates the languages used in each of the three models.

Figure 1.5
The Three Dynamic Semantic Models



In our diagram, we placed axiomatic and denotational semantics above the imperative language whose meaning they describe, using predicate and lambda calculus respectively. Because both predicate and lambda calculi are branches of mathematical logic, they are more abstract than an imperative language. We placed operational semantics below because it defines the meaning of an imperative language by using the lower-level intermediate code into which a compiler would translate it.

A. Operational Semantics

Operational semantics describes the meaning of the statements in a programming language by describing what the statements do. Providing a flowchart for an if statement is really an operational semantic description of its meaning. A less

graphical way to convey the same idea is to define a small set of intermediate code instructions. We need only a small number of instructions. The first set consists of four instructions needed for computation.

```
a ← b                      assignment with no operator
a ← <un_arith_op> b        assignment with a unary arithmetic operator
a ← b <bin_arith_op> c     assignment with a binary arithmetic operator
a ← b(c)                   function call
```

What is most important is that these instructions do not allow nested subexpressions. Each instruction contains no more than a single operator. Assignments containing nested expressions in high-level languages must be translated into these single-instruction assignments.

Figure 1.6 illustrates the generation of intermediate code for an assignment in C or C++ to a nested expression that computes one root of a quadratic equation from its three coefficients.

Figure 1.6
Translation of Expression That Calculates One Root
of a Quadratic Equation into Intermediate Code

0:00 / 1:39

The second set consists of two instructions that perform transfer of control:

```
if a <rel_op> b goto L
go to L
```

These two instructions are usually referred to as a conditional and an unconditional *goto*, respectively. They allow us to define the behavior of high-level control statements that can be nested. Table 1.3 shows how an if and a while statement can be translated into intermediate code.

Note: The symbol ¬ represents negation.

Table 1.3
Translation of Structured Statements into Intermediate Code

High-Level Statement	Intermediate Code

if (<exp>)
  <stmt$_1$>
else
  <stmt$_2$>
	if ¬<exp> goto L_1
	  <stmt$_1$>
	  goto L_2
	L_1 <stmt$_2$>
	L_2 ...

while (<exp>) <stmt>	L_1 if ¬<exp> goto L_2 <stmt> goto L_1 L_2 ...

To illustrate how these translations would be applied, consider the following code segment written in C or C++ that determines whether the value in n is a prime number:

```
i = 2;
while (i <= srqt(n))
{
  if (n % i == 0)
    isPrime = false;
  i++;
}
```

Shown below is the intermediate code that corresponds to the above code segment:

```
      i = 2;
      isPrime = true
      t1 = sqrt(n)
L1    if (i > t1) goto L3
      t2 = n % i
      if (t2 != 0) goto L2
      isPrime = false
L2    i = i + 1
      goto L1
L3    ...
```

The actual target of the *goto* statements must be filled in once the destination statements have been generated. This process is known as *backpatching*.

B. Axiomatic Semantics

Axiomatic semantics defines the meaning of programs using assertions written in predicate calculus called preconditions and postconditions. As such, it defines what must be true before and after a program executes but does not specify how it is to be accomplished. Axiomatic semantics was developed as a part of an effort to prove programs correct. The kind of statement that we prove using axiomatic semantics is one like the following:

{P} S {Q}

This statement claims that if the precondition P is true and program S is executed, postcondition Q will be true.

Now let's consider how axiomatic semantics defines the meaning of statements in an imperative language. We express the meaning of an assignment statement, using the following substitution rule:

$\{Q_{v \rightarrow e}\}\ v = e\ \{Q\}$

In the above rule, $Q_{v \rightarrow e}$ represents the assertion Q having substituted every occurrence of *v* with *e*. Using the substitution rule, we can compute a precondition given a postcondition. In figure 1.7, we demonstrate how a precondition can be computed for the standard three-line code sequence that swaps two variables.

Figure 1.7
Computing the Precondition of a Swap-Code Segment
Using the Substitution Rule

0:00 / 0:58

Compound statements are defined using inference rules. The inference rule for an if statement requires that the expected two cases be proven to draw an inference about the entire statement.

$$\frac{\{P \wedge B\}\ S_1\ \{Q\},\ \{P \wedge \neg B\}\ S_2\ \{Q\}}{\therefore\ \{P\}\ \texttt{if}\ (B)\ S_1\ \texttt{else}\ S_2\ \{Q\}}$$

To illustrate the use of the inference rule for an if statement, let's embed the swap-code segment that we discussed earlier inside an if statement to construct a C++ function that sorts two values into ascending order. That function is shown below with the precondition, postcondition, and all the intervening assertions, numbered and highlighted.

```
// 1 {true}
void sortTwo(int& x, int& y)
{
  if (x > y)
  {
    // 2 {x > y}
    t = x;
    // 3 {t > y}
    x = y;
    // 4 {t > x}
    y = t;
    // 5 {y > x}
  }
  // 6 {x <= y}
  else
    // 7 {x <= y}
    ;
  // 8 {x <= y}
}
// 9 {x <= y}
```

Notice that the precondition is {true}, which is the weakest possible precondition. It is always desirable to state the weakest precondition for a given program, although that weakest precondition is sometimes an assertion stronger than simply {true}.

When we wish to prove the correctness of a function such as sortTwo, we work from the inside out, first establishing that if assertion 2, {x > y}, is true before the three-line swap, assertion 5, {y > x} will be true afterward. We establish that fact using the same technique of substitution that we used earlier. Unfortunately, assertion 5 is not exactly what we need. What we need is assertion 6, {x <= y}. That requires an additional inference rule, called the *rule of consequence,* shown below:

$$\frac{\{P\}\ S\ \{Q\},\ P' \Rightarrow P,\ Q' \Rightarrow Q}{\therefore\ \{P'\}S\{Q'\}}$$

Here is how we apply the above rule to establish the assertion, which is numbered 6.

$$\frac{\{x > y\}\ \texttt{swap}\ \{y > x\},\ y > x \Rightarrow x <= y}{\therefore\ \{x > y\}\ \texttt{swap}\ \{x <= y\}}$$

For the clause Q' ⇒ Q in the rule of consequence, we used an implication we know to be true from arithmetic:

y > x ⇒ x <= y

We did not need the clause P' ⇒ P.

To complete the proof, we must apply the inference rule for an if statement, which we introduced earlier. The application of that rule is shown below:

$$\frac{\{\text{true} \wedge x > y\}\ \texttt{swap}\ \{x <= y\},\ \{\text{true} \wedge x <= y\}\ ;\ \{x <= y\}}{\therefore\ \{\text{true}\}\ \texttt{if}\ (x > y)\ \texttt{swap else}\ ;\ \{x <= y\}}$$

We have completed the proof and can now state {true} sortTwo {x <= y}.

The inference rule for a while statement requires that we find a loop invariant, which in the following inference rule is the assertion P.

$$\frac{\{P \wedge B\}\ S\ \{P\}}{\therefore\ \{P\}\ \texttt{while}\ (B)\ S\ \{P \wedge \neg B\}}$$

Finding loop invariants is often the most challenging aspect of correctness proofs. Because proofs involving loops can become quite complicated, we omit them.

Perhaps the most important idea to understand about axiomatic semantics is that it defines the meaning of a program in a language that is more abstract than the way in which it is expressed in an imperative language. An axiomatic statement of meaning suggests no algorithm. As an illustration of that point, consider the fact that a bubble sort, quick sort, heaps sort, and so on would all have identical pre- and postconditions.

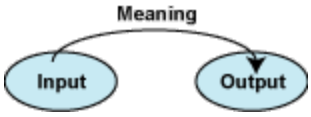
The fact that both axiomatic semantics and logic programming languages are based on predicate calculus suggests a strong connection between them. We will get a better sense of that connection in module 5.

C. Denotational Semantics

As we mentioned in our introduction to dynamic semantics, denotational semantics, like axiomatic semantics, is based on a branch of mathematical logic—in this case, lambda calculus. Lambda calculus is also referred to as *recursive function theory*.

The meaning of a program from the perspective of denotational semantics is a mathematical function that maps the input of the program as the domain to the output as the codomain. Figure 1.8 illustrates this idea.

Figure 1.8
The Meaning of a Program as a Mathematical Function



The meaning of individual statements of an imperative language can be viewed as functions also, but the domain and codomain of those functions are the state of the machine. Understanding this view of statement meaning will help you later to understand the distinction between imperative and declarative languages. When we encounter functional languages in module 5, we will see that programs in functional languages, unlike programs in imperative languages, are stateless.

To help explain this view of program meaning, we will begin with a computational mechanism involving only assignment statements, similar to what we did with axiomatic semantics. When no input or output is involved, we can capture the denotational meaning of a block of code using a concurrent assignment.

Figure 1.9 illustrates this method of computing a concurrent assignment, again using the three-line swap-code segment as our example.

Figure 1.9
Computing the Concurrent Assignment of a Swap-Code Segment

0:00 / 0:58

If we are dealing with three integer variables, the state space at that point in the program is the set:

$$\mathbf{Z} \times \mathbf{Z} \times \mathbf{Z}$$

The meaning of each assignment then is a function a:

$$\mathbf{Z} \times \mathbf{Z} \times \mathbf{Z} \rightarrow \mathbf{Z} \times \mathbf{Z} \times \mathbf{Z}$$

which is exactly what is represented by a concurrent assignment of three variables. Suppose that a_1, a_2, and a_3 represent the functional meanings of three successive assignments, such as the ones in our three-line swap. The meaning of the whole block is the composition of the three functions:

$$a_1 \circ a_2 \circ a_3$$

This function composition is exactly what we computed using the technique of symbolic execution illustrated in figure 1.10.

Function composition is exactly how the meaning of a statement list is defined. Assume that *M* is the function that converts a statement to its functional meaning, and that S_1 S_2 ... S_n are the component statements of a statement list, and that *s* is the state. Then, we can define the meaning of a statement list as follows:

$$M(S_1\ S_2 \ldots S_n)\ (s) = M(S_n) \circ \ldots \circ M(S_2) \circ\ M(S_1)\ (s)$$

Next, let's consider how we can define the function meaning of an if statement.

$$M(\text{if (B) } S_1 \text{ else } S_2)\ (s) = \begin{cases} M(S_1) \text{ if } B(s) \\ M(S_2) \text{ otherwise} \end{cases}$$

This definition of the meaning of an if statement should be easier to grasp than the meaning of the statement list. It says that the meaning of an if statement is the meaning of the first statement S_1 if B is true in state *s*, and it is the meaning of the second statement S_2 otherwise.

If we combine our technique of computing a concurrent assignment for a sequence of assignments with the definition of the meaning of an if statement, we can state the functional meaning of the C++ function sortTwo, which we used as an example in our discussion of axiomatic semantics, as follows:

$$M(\text{sortTwo}) = \begin{cases} x, y = y, x, \text{ if } x > y \\ x, y = x, y, \text{ otherwise} \end{cases}$$

There are two aspects of this definition that warrant comment. First, we have dropped the local temporary variable t from our concurrent assignment. From the perspective of a whole C++ function, the mathematical function that defines its meaning needs only to involve the parameters, never the local variables. Although the local variables are a part of intermediate states internal to the function, they are not a part of the initial or final states of the function as a whole.

The second aspect is the concurrent assignment in the "otherwise case." The code that was executed in that case was the null statement. Its meaning is the identity function, which is a concurrent assignment in which each *n*-tuple is mapped to

itself.

We conclude by defining the functional meaning of a while statement, which is shown below:

$$M(\text{while (B) S})(s) = \begin{cases} M(\text{while (B) S}) \circ M(\text{S})(s) & \text{if B}(s) \\ s, & \text{otherwise} \end{cases}$$

As was true in the case of axiomatic semantics, understanding the functional definition of the meaning of the while statement is a bit more difficult, so we will forgo any examples, and just make a few important observations about this definition. It is a recursive definition with two cases—the first being the recursive case and the second being the base case. In the recursive case, the meaning involves composition of the function defining the whole loop with the meaning of the statement that constitutes the body of the loop. Finally, observe that the function being defined in the base case is the identity function.

We recognize that dynamic semantics is a rather abstract topic. The language of mathematical logic used in both cases is a declarative language. As with declarative programming languages, recursion must be used to express what would be expressed iteratively in an imperative language.

Finally, note that both denotational semantics and functional programming languages are based on lambda calculus. In fact, denotational semantics is sometimes called *functional semantics*. In the case study for module 5, we will present an interpreter for an imperative language that is written in a functional language. For each statement of that imperative language, the interpreter will contain a function that essentially defines the functional meaning of that statement. At that time, we will encourage you to reread this section to see how closely related the two really are.

V. Syntax Translation

As we discussed earlier in this module, the first two phases of the compiler deal with verifying that the program submitted has correct syntax. The scanner or lexical analyzer handles the lexical level that breaks the character stream into lexemes and

categorizes those lexemes into tokens. The parser or syntactic analyzer handles the textual level, which includes each of the remaining levels from expressions to statements to subprograms, and in object-oriented languages, classes or some similar construct.

For a variety of reasons, it is customary to divide syntax into these two levels. We mentioned one of those earlier, which is that because it is easy to restrict token syntax to disallow nesting, token syntax can be defined with regular expressions—a less powerful defining mechanism than BNF. Another important difference between these two levels in most high-level languages is the role of white space, which includes spaces, tabs, and new-line characters. Whereas any amount of white space is permitted between lexemes without affecting the program meaning, white space cannot be arbitrarily inserted between characters within tokens without changing the program.

A. Scanners

In most modern compilers, scanners are not hand-coded. Because the theory that surrounds regular expressions and finite state automata is so well understood, there are programs, called *lexical analyzer generators,* that accept the regular expression definition of the tokens of a programming language and produce the transition table for the finite state automata that accepts those tokens. One of the most popular lexical analyzer generators is one called lex that is a part of the UNIX operating system.

It is, of course, possible to hand-code a lexical analyzer in a high-level language such as Java. In the case study for this module, we will examine an example of one such scanner.

B. Parsers

Parsers are, of necessity, more complicated programs than scanners because they must be able to recognize languages whose syntax involves nesting, as is universally true among modern high-level languages. Nonetheless, like recognizing languages defined with regular expressions, the problem of recognizing context-free languages is so well understood that programs called *syntactic analyzer*

generators exist that produce parse tables that are used by a very simple algorithm. These algorithms implement a machine called a *pushdown automaton*. One such generator, named yacc, is provided with the UNIX operating system.

Types of Parsers

There are fundamentally two kinds of parsers. The first kind is the *top-down parser,* which constructs the parse tree from the root start symbol to the terminal symbols that are the leaves. Figure 1.3, which we encountered earlier, constructed the parse tree in this fashion. The second kind is the *bottom-up parser,* which begins with the leaves and works toward the root. Bottom-up parsers are more powerful than top-down ones, which means they are capable of recognizing a larger number of grammars.

We intend to focus exclusively on top-down parsers, which although less powerful, are somewhat simpler. We leave bottom-up parsers, of which yacc is an example, to a more advanced course on language translation.

The problem of efficiently parsing languages that are defined by arbitrary grammars is difficult because for some grammars, it may be impossible at each step to know the correct production to choose, making it necessary to backtrack. Because of the inefficiency incurred by backtracking, the efficiency of parsing algorithms that can work with any grammar is $O(n^3)$, where n is the length of the program to be parsed. By restricting allowable grammars, algorithms can be developed that are $O(n)$, which is clearly far more desirable.

The family of grammars that can be efficiently parsed with top-down algorithms are called *LL(1) grammars*. Two restrictions must be imposed for a grammar to be able to be parsed efficiently top-down:

1. No left-recursive productions are permitted.
2. In a given state, we can always know what production to choose by looking at the next token.

There are two kinds of top-down parsers—those that use parse tables generated by a syntactic analyzer generator and hand-coded parsers called *recursive descent parsers*. Our focus will be exclusively on the latter.

Recursive Descent Parser

The rules for constructing a recursive descent parser are quite simple. There must be a function for each production, which will be recursive if the production is recursive. When writing the function to parse a particular production, the symbols on the right-hand side are processed one-by-one. If the symbol is a nonterminal, a call is made to the function that parses the production with that nonterminal on the left-hand side. If it is a terminal symbol, the function verifies that the next token in the input stream is that token. If so, it consumes it; if not, a syntax error is generated.

Clearly, an example best illustrates how a recursive descent parser is constructed. Let's consider a few productions from the grammar of Pascal.

```
<compound_stmt> → begin <stmt_list> end
<stmt_list> → <stmt> ; <stmt_list> | <stmt>
<stmt> → <compound_stmt> | <if_stmt>
<if_stmt> → if <exp> then <stmt> [else <stmt>]
```

Let's begin with a few observations about the above grammar. It contains a pair of indirectly recursive productions. An if statement contains statements, and a statement can be another if statement. Note, also, that the first choice on the right-hand side of the statement-list production is right-recursive.

Now let's examine the Java methods that would be needed to parse the first three productions, beginning with the compound statement.

Move your mouse over the highlighted parts of the method to see the explanations of those parts.

```
boolean compoundStmt()
{
    if (token == BEGIN)
    {
        token = getNextToken();
        if (stmtList())
            if (token == END)
            {
                token = getNextToken();
                return true;
            }
    }
    return false;
}
```

Next, let's consider the method that parses statements.

Move your mouse over the highlighted part of the method to see the explanations of that part.

```
boolean stmt()
{
    if (token == IF)
        return ifStmt();
    if (token == BEGIN)
        return compoundStmt();
    return false;
}
```

Finally, let's consider the method that parses a statement list.

Move your mouse over the highlighted parts of the method to see the explanations of those parts.

```
boolean stmtList()
{
    if (stmt())
        if (token == ';')
        {
            token = getNextToken();
            return stmtList();
        }
        else
            return true;
    return false;
}
```

In the way we have written the parser, the method for parsing the complete program will ultimately return true or false—either the program parsed, or it contained a syntax error. Parsers in production compilers must do more than just report success or failure. They must locate and identify the type of error, then recover from the errors to attempt to find other errors. Error recovery is one of the most challenging aspects of writing a parser. In module 3, we will discuss how the design of the grammar of a language can make the recovery process easier.

In the case study for this module, we present a complete program that contains a recursive descent parser.

Return to top of page

© 2026 University of Maryland Global Campus

All links to external sites were verified at the time of publication. UMGC is not responsible for the validity or integrity of information located at external sites.